```
if (!state.hasTimedOut) {
  val transactionList = new ListBuffer[InputRow]()
  values.foreach { value =>
    transactionList.append(value)}

  var prevState = state.getOption.getOrElse {
    val firstTransactionTimestamp = transactionList.head.
    TransactionTimestamp
    new TransactionCountState(firstTransactionTimestamp, List
    [InputRow]())}

  val stateWithNewRecords = addNewRecords(transactionList.toList,
    prevState)

  val stateWithRecordsDropped =
    dropExpiredRecords(stateWithNewRecords.latestTimestamp,
    stateWithNewRecords.currentTransactions)

  val output = new TransactionCount(CustomerID,
    stateWithRecordsDropped.currentTransactions.size,
    stateWithRecordsDropped.latestTimestamp, false)

  transactionCounts.append(output)
  state.update(stateWithRecordsDropped)
  state.setTimeoutTimestamp(
    stateWithRecordsDropped.latestTimestamp.toEpochMilli(),
    "30 seconds")
} else {
```

Figure 5.19 – If the state has not timed out, the updateState function receives and processes records

On the other hand, as shown in *Figure 5.20*, if no records are received for a given `CustomerID` within a minute since the last time this function was called, it will drop any records that are older than `windowMinutes` from the state and adjust the count: